Faculty of Science and Engineering
School of Computing and Information Science

MOD006568
Neural Computing & Deel Learning

Element 010
Logbook

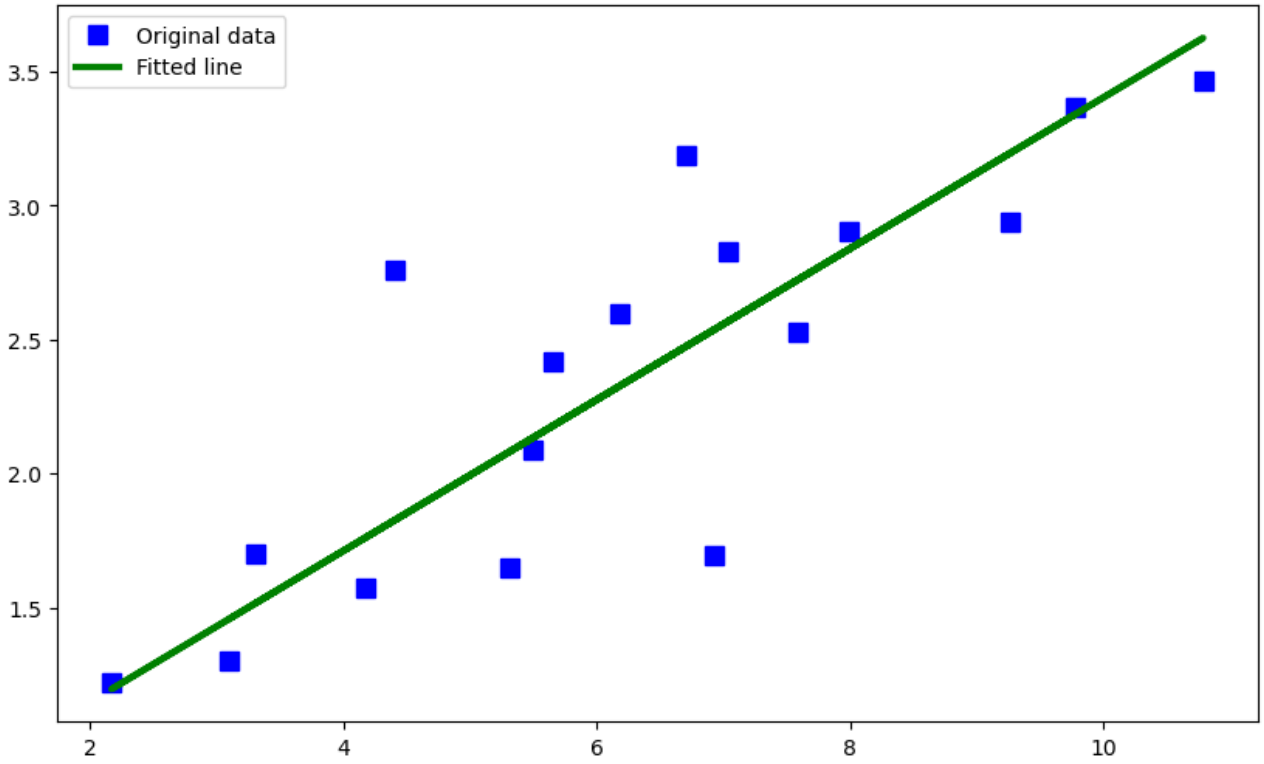
SID NUMBER: 2128741

Github Link: https://github.com/jugal-chawda/MOD006568-Neural-Computing-and-Deep-Learning

Submission deadline: 01/05/2026

LAB Logbook

Lab1



Lab 2

```
functional_model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

```
functional_history = functional_model.fit(X_train, y_train, epochs=200, batch_size=16, verbose=1)
```

```
Epoch 172/200
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 1.0000 - loss: 0.0023
Epoch 173/200
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 1.0000 - loss: 0.0020
Epoch 174/200
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 1.0000 - loss: 0.0018
Epoch 175/200
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 1.0000 - loss: 0.0018
Epoch 176/200
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 1.0000 - loss: 0.0017
Epoch 177/200
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 1.0000 - loss: 0.0017
Epoch 178/200
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 1.0000 - loss: 0.0019
Epoch 179/200
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 1.0000 - loss: 0.0016
Epoch 180/200
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 1.0000 - loss: 0.0016
Epoch 181/200
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 1.0000 - loss: 0.0015
Epoch 182/200
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 1.0000 - loss: 0.0019
Epoch 183/200
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 1.0000 - loss: 0.0024
Epoch 184/200
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 1.0000 - loss: 0.0018
Epoch 185/200
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 1.0000 - loss: 0.0014
Epoch 186/200
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 1.0000 - loss: 0.0013
Epoch 187/200
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 1.0000 - loss: 0.0014
Epoch 188/200
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 1.0000 - loss: 0.0013
Epoch 189/200
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 1.0000 - loss: 0.0015
Epoch 190/200
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 1.0000 - loss: 0.0019
Epoch 191/200
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 1.0000 - loss: 0.0017
Epoch 192/200
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 1.0000 - loss: 0.0012
Epoch 193/200
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 1.0000 - loss: 0.0013
Epoch 194/200
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 1.0000 - loss: 0.0013
Epoch 195/200
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 1.0000 - loss: 0.0012
Epoch 196/200
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 1.0000 - loss: 0.0013
Epoch 197/200
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 1.0000 - loss: 0.0010
Epoch 198/200
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 1.0000 - loss: 0.0012
Epoch 199/200
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 1.0000 - loss: 0.0011
Epoch 200/200
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 1.0000 - loss: 9.9926e-04
```

```
loss, acc = functional_model.evaluate(X_test, y_test, verbose=1)
print('Functional Model Test Accuracy: %.3f' % acc)
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9052 - loss: 0.5566
Functional Model Test Accuracy: 0.905
```

Lab 3

[39]
0s

```
test_loss, test_acc = model.evaluate(test_images, test_labels, verbose=2)
print('Test accuracy:', test_acc)
```

```
313/313 - 1s - 2ms/step - accuracy: 0.8891 - loss: 0.3984
Test accuracy: 0.8891000151634216
```

[40]
0s

```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 10)	650

Total params: 328,160 (1.25 MB)
Trainable params: 109,386 (427.29 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 218,774 (854.59 KB)

Lab 4

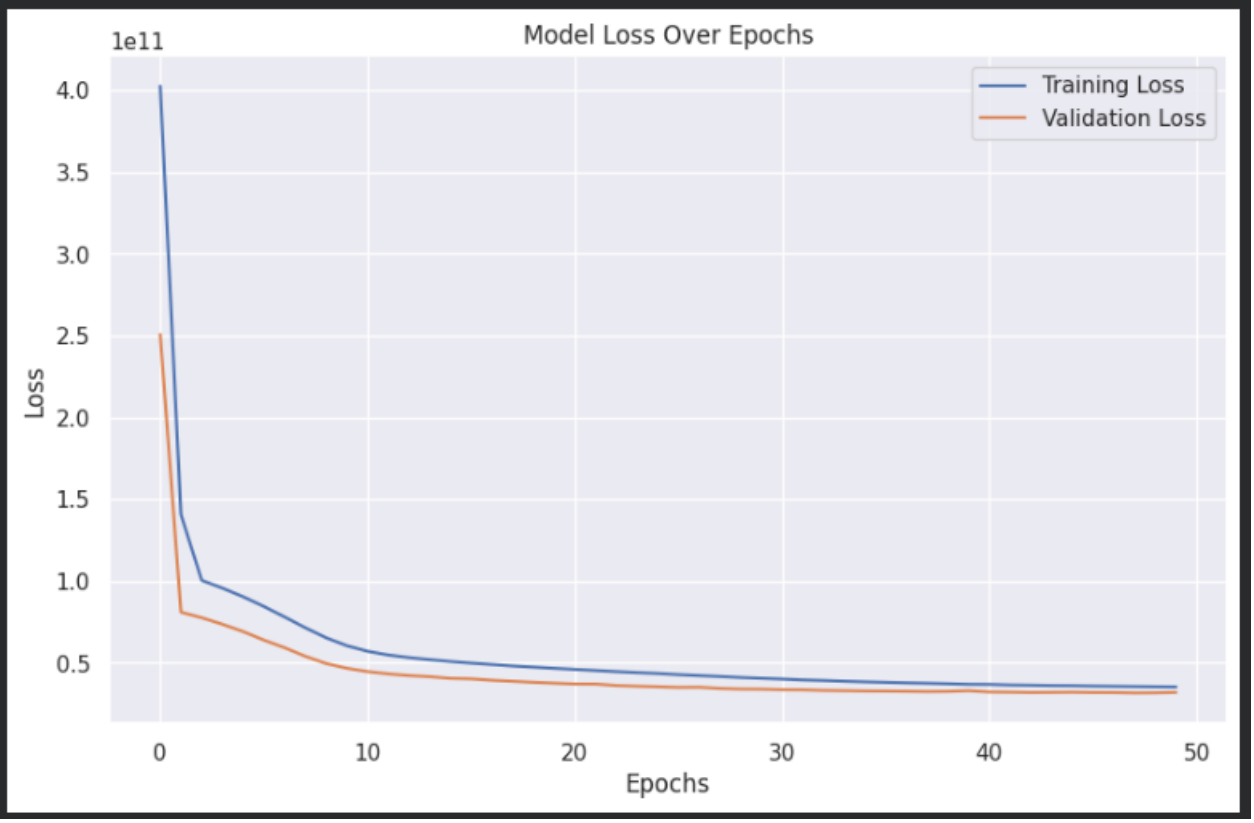
```
model.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
dense_9 (Dense)	(None, 128)	2,560
dense_10 (Dense)	(None, 64)	8,256
dense_11 (Dense)	(None, 32)	2,080
dense_12 (Dense)	(None, 1)	33

Total params: 38,789 (151.52 KB)
Trainable params: 12,929 (50.50 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 25,860 (101.02 KB)

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10,6))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.title('Model Loss Over Epochs')
plt.show()
```



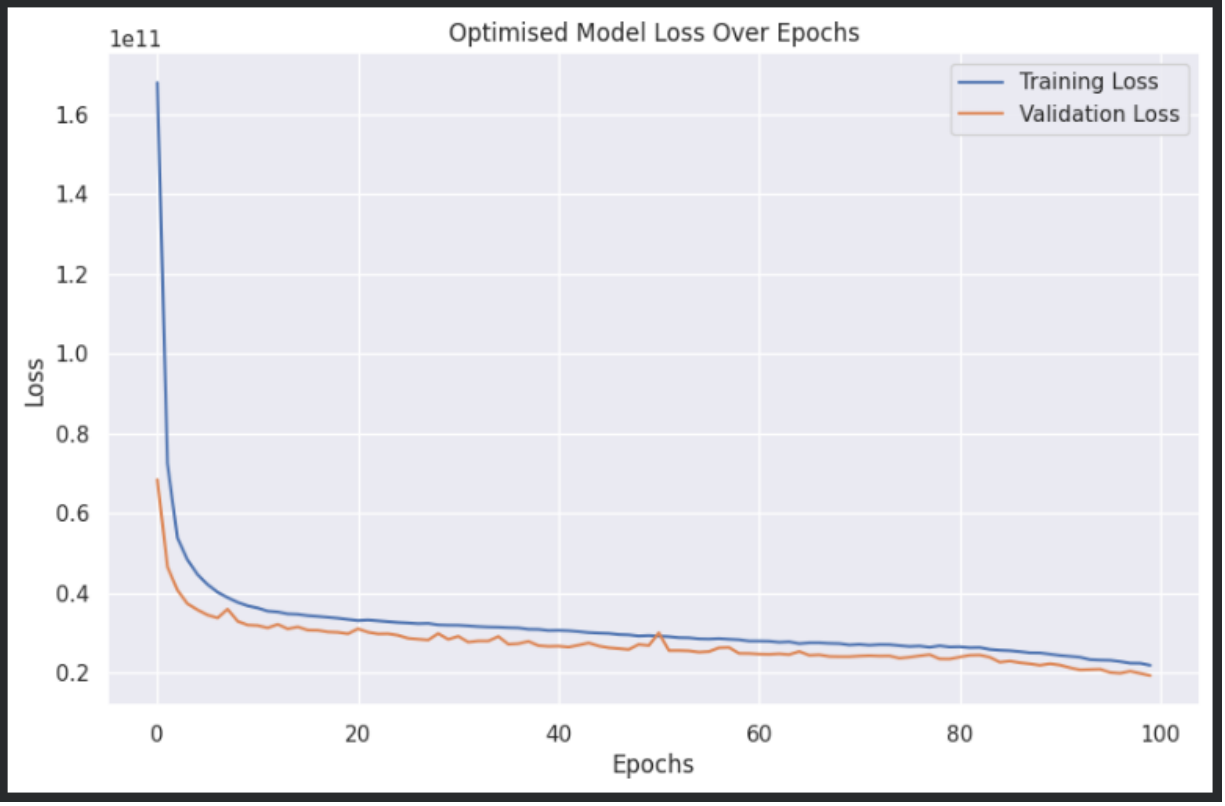
```
model.summary()
```

Model: "sequential_3"

Layer (type)	Output Shape	Param #
dense_13 (Dense)	(None, 256)	5,120
dense_14 (Dense)	(None, 128)	32,896
dense_15 (Dense)	(None, 64)	8,256
dense_16 (Dense)	(None, 32)	2,080
dense_17 (Dense)	(None, 1)	33

Total params: 145,157 (567.02 KB)
Trainable params: 48,385 (189.00 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 96,772 (378.02 KB)

```
plt.figure(figsize=(10,6))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.title('Optimised Model Loss Over Epochs')
plt.show()
```



Lab 5

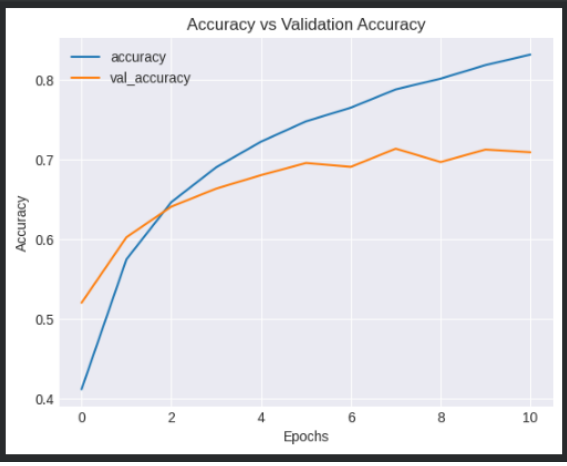
```
# Model building code

model.summary()
```

Model: "sequential"

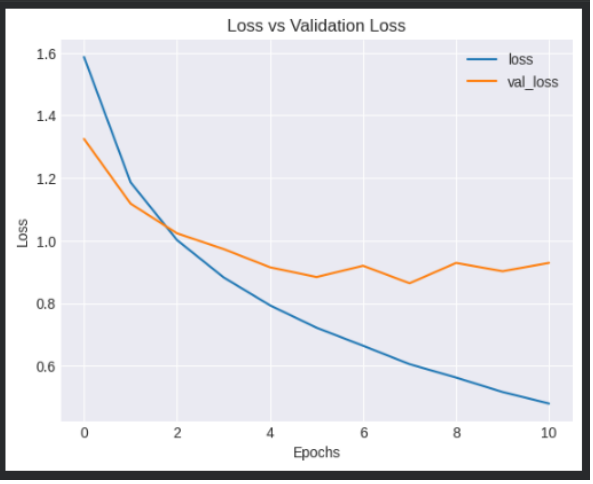
Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 29, 29, 32)	1,568
max_pooling2d (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_1 (Conv2D)	(None, 12, 12, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 64)	0
conv2d_2 (Conv2D)	(None, 4, 4, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 2, 2, 128)	0
flatten (Flatten)	(None, 512)	0
dense (Dense)	(None, 128)	65,664
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 10)	650

Total params: 505,472 (1.93 MB)
Trainable params: 168,490 (658.16 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 336,982 (1.29 MB)

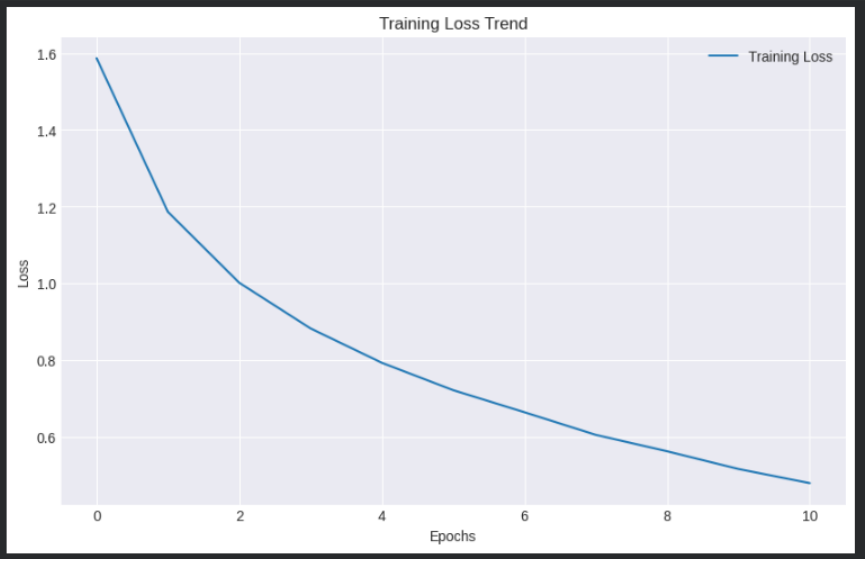
```
# Graph 1: Accuracy and validation accuracy

losses[['accuracy', 'val_accuracy']].plot()
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.title('Accuracy vs Validation Accuracy')
plt.show()
```



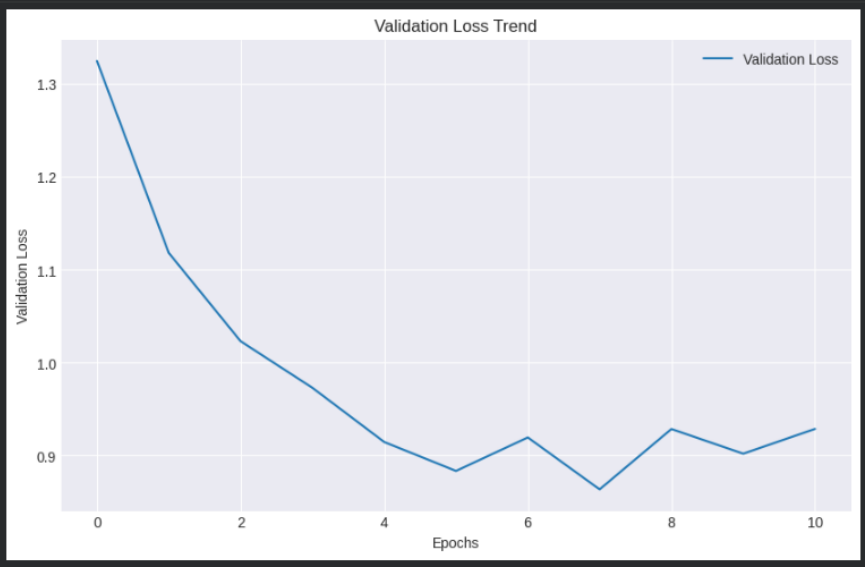

```
# Graph 2: Loss and validation loss

losses[['loss', 'val_loss']].plot()
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Loss vs Validation Loss')
plt.show()
```



```python
# Graph 3: Training loss trend

plt.figure(figsize=(10,6))
plt.plot(history.history['loss'], label='Training Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Training Loss Trend')
plt.legend()
plt.show()
```




```python
# Graph 4: Validation loss trend

plt.figure(figsize=(10,6))
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Validation Loss')
plt.title('Validation Loss Trend')
plt.legend()
plt.show()
```

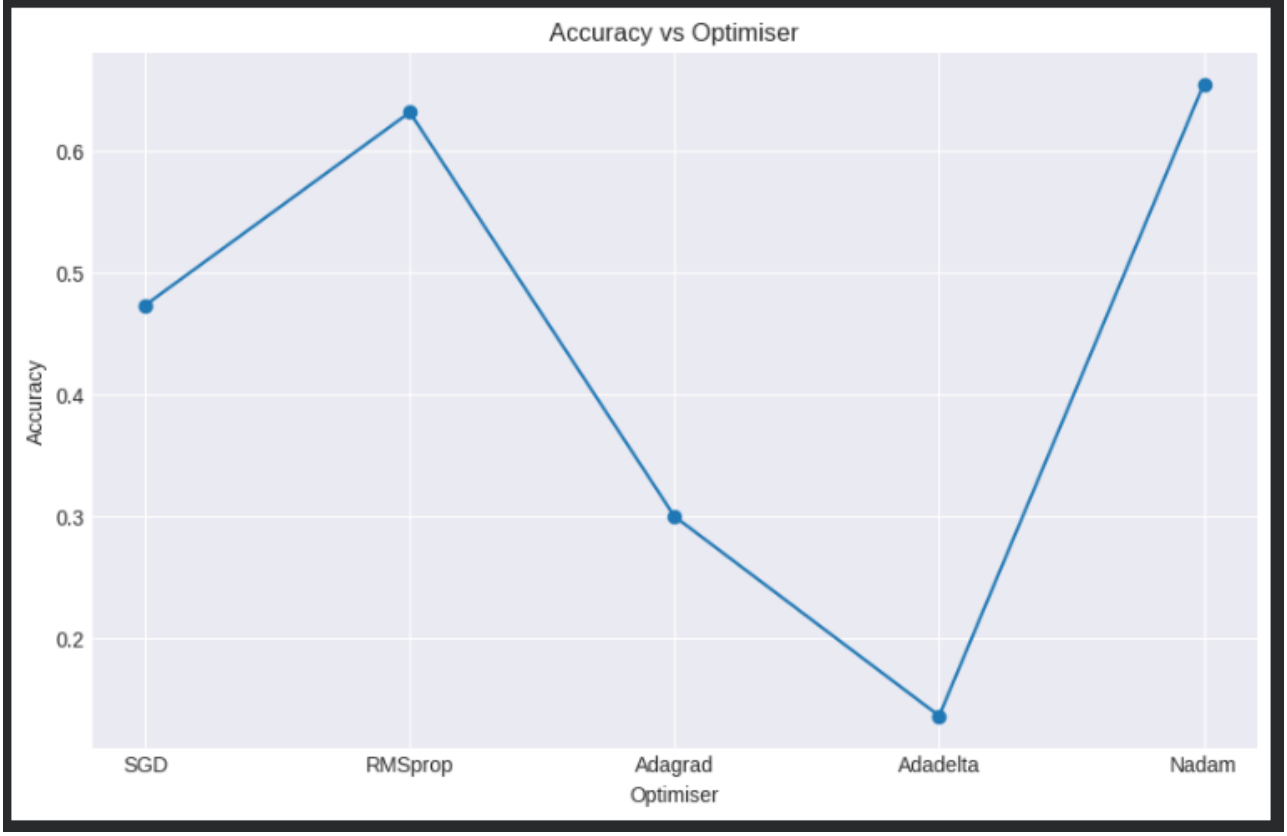



```python
# Optimal number of epochs based on minimum validation loss

optimal_epoch = np.argmin(history.history['val_loss']) + 1
print("Optimal number of epochs:", optimal_epoch)
```

```
Optimal number of epochs: 8
```

Optional

```
# Plot score vs optimiser

plt.figure(figsize=(10,6))
plt.plot(optimisers, scores, marker='o')
plt.xlabel('Optimiser')
plt.ylabel('Accuracy')
plt.title('Accuracy vs Optimiser')
plt.show()
```



Lab 6

```
# Model creation and summary

model_2 = Sequential()
model_2.add(tf.keras.Input(shape=(length, n_features)))
model_2.add(LSTM(100, activation='relu'))
model_2.add(Dense(1))

model_2.compile(optimizer='adam', loss='mse')

model_2.summary()
```

```
Model: "sequential_3"
```

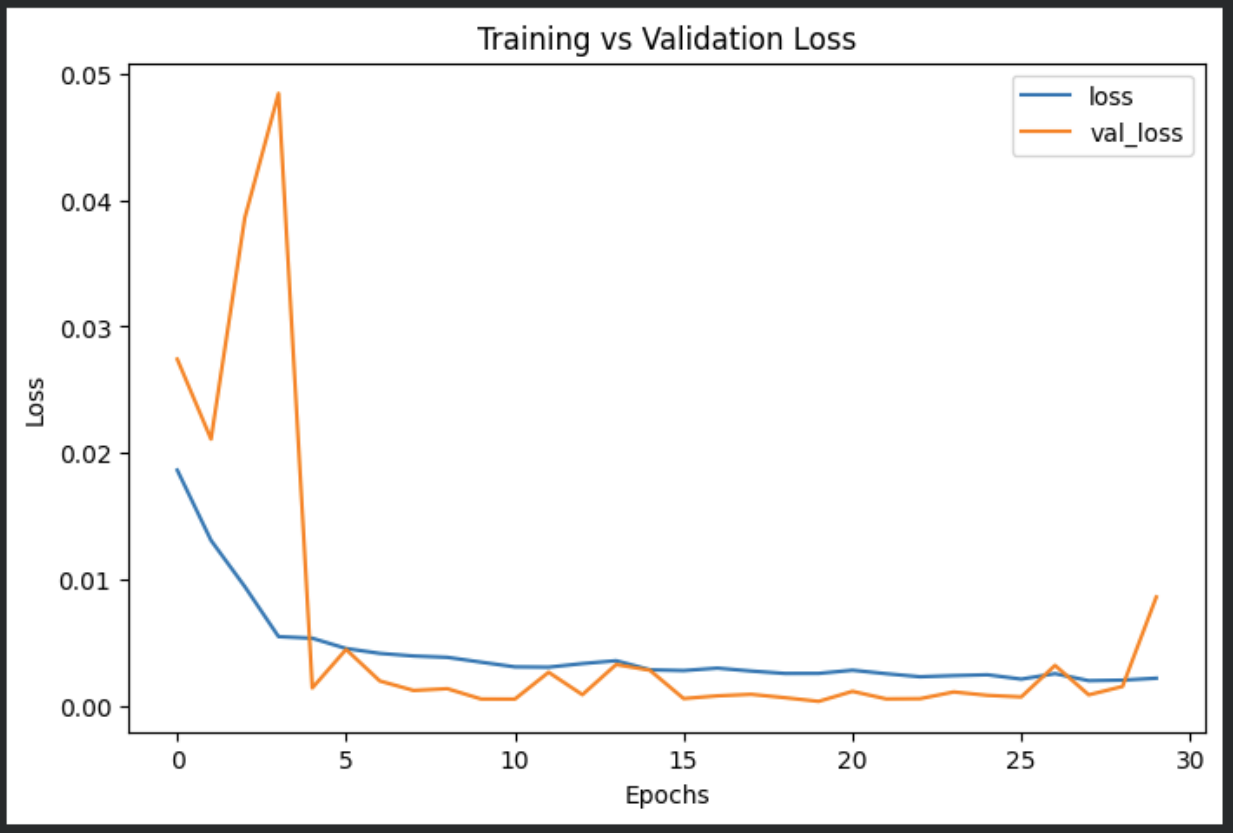
Layer (type)	Output Shape	Param #
lstm_3 (LSTM)	(None, 100)	40,800
dense_3 (Dense)	(None, 1)	101

```
Total params: 40,901 (159.77 KB)
Trainable params: 40,901 (159.77 KB)
Non-trainable params: 0 (0.00 B)
```

```python
# Early stopping and training

early_stop = EarlyStopping(monitor='val_loss', patience=11)

history_2 = model_2.fit(generator,
                        epochs=30,
                        validation_data=validation_generator,
                        callbacks=[early_stop])
```

```
Epoch 2/30
384/384 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.0131 - val_loss: 0.0211
Epoch 3/30
384/384 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0094 - val_loss: 0.0386
Epoch 4/30
384/384 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0055 - val_loss: 0.0485
Epoch 5/30
384/384 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0053 - val_loss: 0.0014
Epoch 6/30
384/384 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0045 - val_loss: 0.0044
Epoch 7/30
384/384 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0041 - val_loss: 0.0019
Epoch 8/30
384/384 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0039 - val_loss: 0.0012
Epoch 9/30
384/384 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.0038 - val_loss: 0.0013
Epoch 10/30
384/384 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.0034 - val_loss: 5.1192e-04
Epoch 11/30
384/384 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0031 - val_loss: 5.0338e-04
Epoch 12/30
384/384 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0030 - val_loss: 0.0026
Epoch 13/30
384/384 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0033 - val_loss: 8.5436e-04
Epoch 14/30
384/384 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0035 - val_loss: 0.0032
Epoch 15/30
384/384 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0028 - val_loss: 0.0028
Epoch 16/30
384/384 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0028 - val_loss: 5.4653e-04
Epoch 17/30
384/384 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.0030 - val_loss: 7.6188e-04
Epoch 18/30
384/384 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.0027 - val_loss: 8.8164e-04
Epoch 19/30
384/384 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0025 - val_loss: 6.1584e-04
Epoch 20/30
384/384 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0025 - val_loss: 3.2517e-04
Epoch 21/30
384/384 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0028 - val_loss: 0.0011
Epoch 22/30
384/384 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0025 - val_loss: 5.1962e-04
Epoch 23/30
384/384 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0023 - val_loss: 5.3560e-04
Epoch 24/30
384/384 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0024 - val_loss: 0.0011
Epoch 25/30
384/384 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.0024 - val_loss: 8.0266e-04
Epoch 26/30
384/384 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - loss: 0.0021 - val_loss: 6.7597e-04
Epoch 27/30
384/384 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0025 - val_loss: 0.0032
Epoch 28/30
384/384 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0020 - val_loss: 8.4097e-04
Epoch 29/30
384/384 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0020 - val_loss: 0.0015
Epoch 30/30
384/384 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0022 - val_loss: 0.0086
```

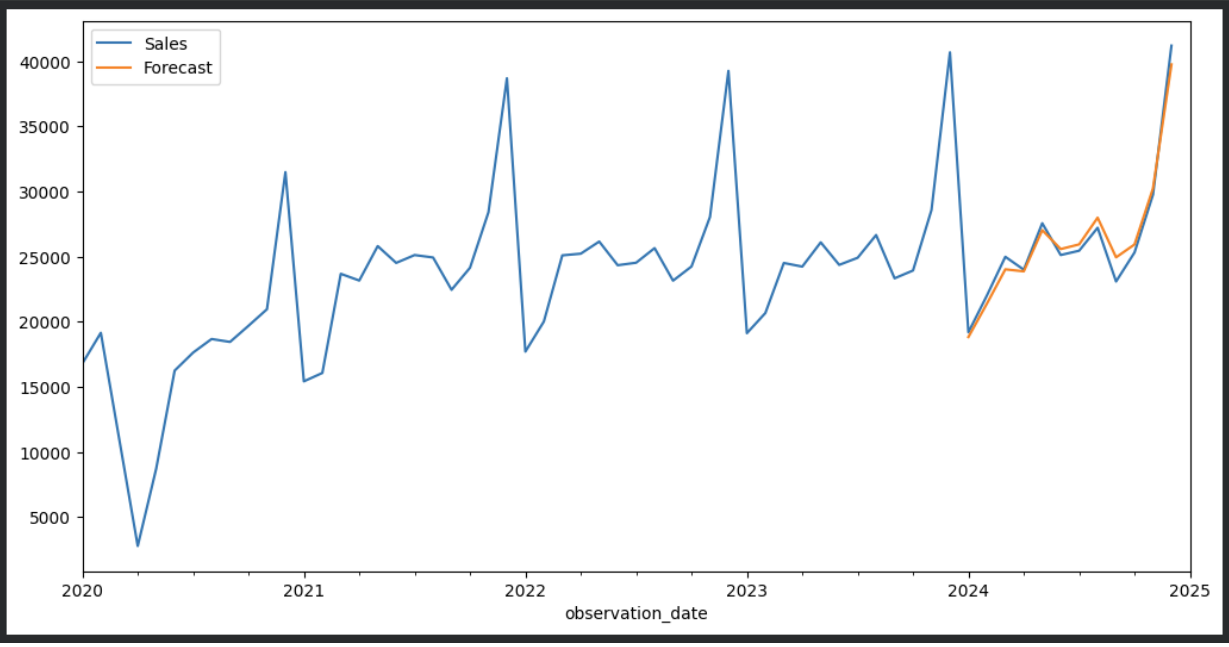
```python
# Plot training and validation loss

losses_2 = pd.DataFrame(history_2.history)

losses_2[['loss','val_loss']].plot(figsize=(8,5))
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Training vs Validation Loss')
plt.show()
```




```python
# Find optimal number of epochs

optimal_epoch = losses_2['val_loss'].idxmin() + 1
print("Optimal number of epochs:", optimal_epoch)
```

```
Optimal number of epochs: 20
```

```python
# Plot forecast for last 12 months

forecast_index = pd.date_range(start='2024-01-01', periods=periods, freq='MS')

forecast_df = pd.DataFrame(data=forecast,
                           index=forecast_index,
                           columns=['Forecast'])

ax = df.plot(figsize=(12,6))
forecast_df.plot(ax=ax)

plt.xlim('2020-01-01','2025-01-01')
plt.show()
```



Optional

```python
# LSTM model creation and summary

lstm_model = Sequential()
lstm_model.add(tf.keras.Input(shape=(length, n_features)))
lstm_model.add(LSTM(100, activation='relu'))
lstm_model.add(Dense(1))

lstm_model.compile(optimizer='adam', loss='mse')

lstm_model.summary()
```

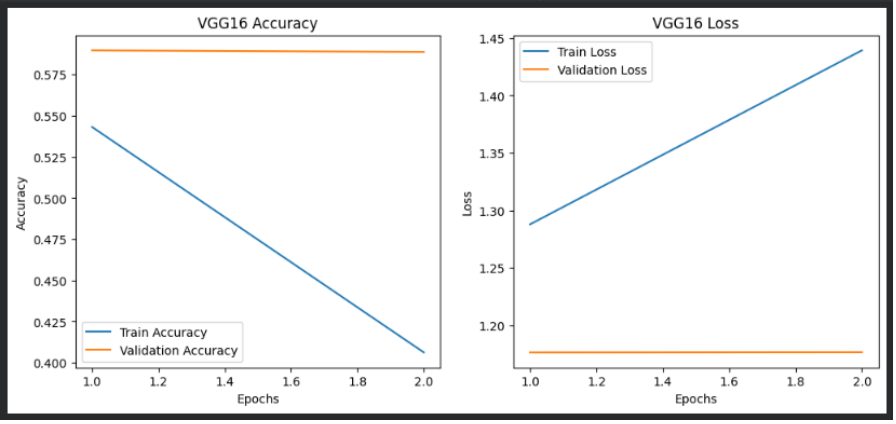
```
Model: "sequential_5"
```

Layer (type)	Output Shape	Param #
lstm_5 (LSTM)	(None, 100)	40,800
dense_5 (Dense)	(None, 1)	101

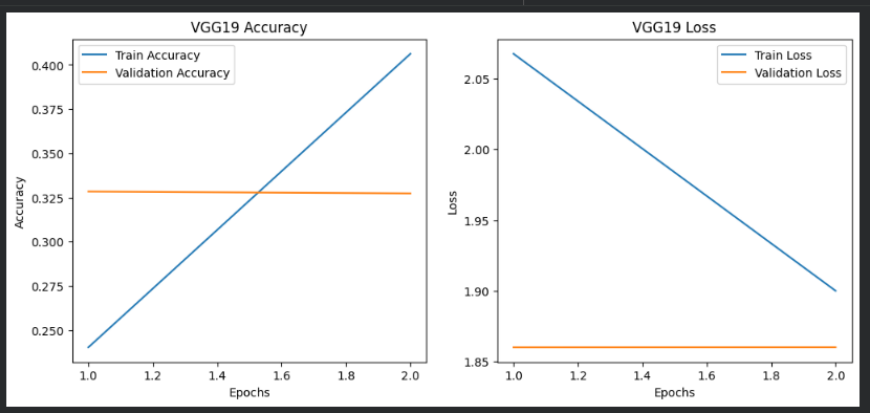
```
Total params: 40,901 (159.77 KB)
Trainable params: 40,901 (159.77 KB)
Non-trainable params: 0 (0.00 B)
```

Lab 7

```python
# Plot Accuracy and Loss for VGG16 model

epochs_vgg16 = range(1, len(history.history['loss']) + 1)

plt.figure(figsize=(12,5))

# Accuracy plot
plt.subplot(1,2,1)
plt.plot(epochs_vgg16, history.history['accuracy'], label='Train Accuracy')
plt.plot(epochs_vgg16, history.history['val_accuracy'], label='Validation Accuracy')
plt.title('VGG16 Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()

# Loss plot
plt.subplot(1,2,2)
plt.plot(epochs_vgg16, history.history['loss'], label='Train Loss')
plt.plot(epochs_vgg16, history.history['val_loss'], label='Validation Loss')
plt.title('VGG16 Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()

plt.show()
```




```python
# Plot Accuracy and Loss for VGG19 model

epochs_vgg19 = range(1, len(history_2.history['loss']) + 1)

plt.figure(figsize=(12,5))

# Accuracy plot
plt.subplot(1,2,1)
plt.plot(epochs_vgg19, history_2.history['accuracy'], label='Train Accuracy')
plt.plot(epochs_vgg19, history_2.history['val_accuracy'], label='Validation Accuracy')
plt.title('VGG19 Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()

# Loss plot
plt.subplot(1,2,2)
plt.plot(epochs_vgg19, history_2.history['loss'], label='Train Loss')
plt.plot(epochs_vgg19, history_2.history['val_loss'], label='Validation Loss')
plt.title('VGG19 Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()

plt.show()
```



```python
# Determine optimal epochs using validation loss minimum

optimal_epoch_vgg16 = history.history['val_loss'].index(min(history.history['val_loss'])) + 1
optimal_epoch_vgg19 = history_2.history['val_loss'].index(min(history_2.history['val_loss'])) + 1

print("Optimal Epoch (VGG16):", optimal_epoch_vgg16)
print("Optimal Epoch (VGG19):", optimal_epoch_vgg19)
```

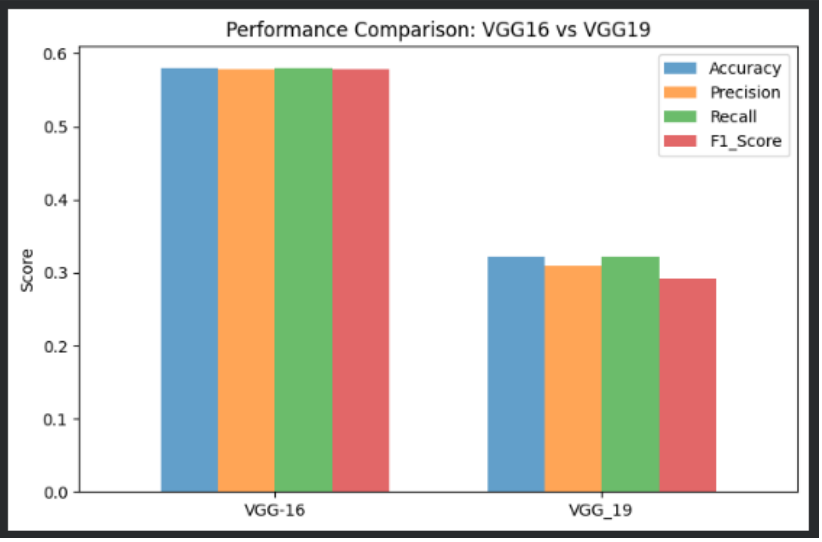
```
Optimal Epoch (VGG16): 1
Optimal Epoch (VGG19): 1
```

```python
# Early stopping configuration based on SID (last digit = 1 → patience = 11)

from tensorflow.keras.callbacks import EarlyStopping

early_stop = EarlyStopping(
    monitor='val_loss',
    patience=11,
    restore_best_weights=True
)
```

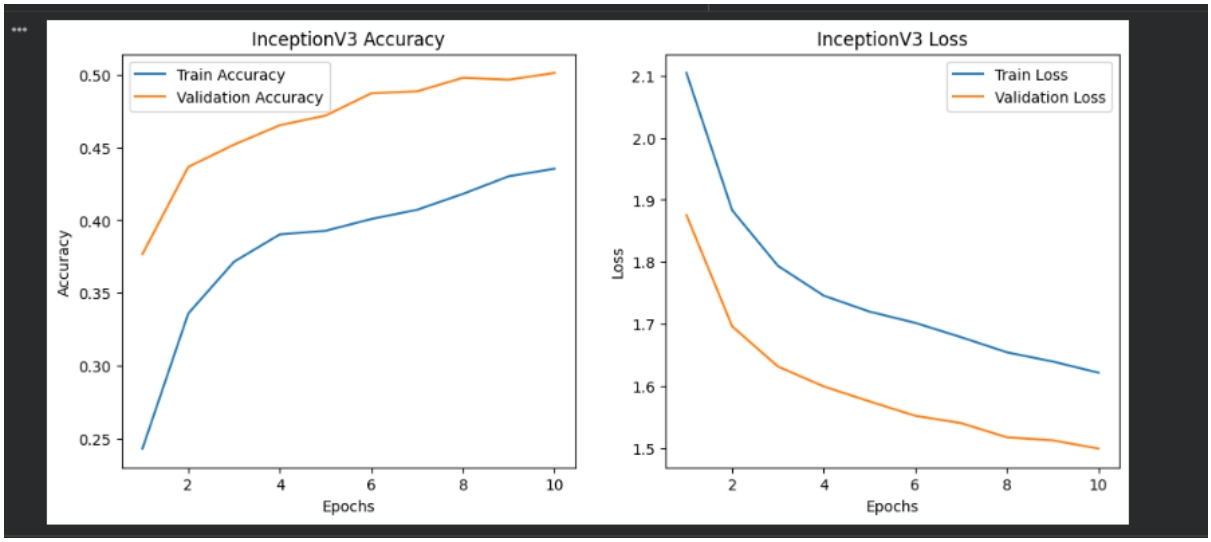
```python
# Create performance DataFrame

ErrorMetrics_pd = pd.DataFrame(ErrorMetrics)
ErrorMetrics_pd.columns = PretrainedModels
ErrorMetrics_pd = ErrorMetrics_pd.T
ErrorMetrics_pd.columns = ['Accuracy','Precision','Recall','F1_Score']

ErrorMetrics_pd
```

	Accuracy	Precision	Recall	F1_Score
VGG-16	0.5807	0.578663	0.5807	0.577993
VGG_19	0.3215	0.309152	0.3215	0.292583

```python
# Plot comparison histogram for VGG16 vs VGG19

ErrorMetrics_pd.plot.bar(width=0.7, alpha=0.7, figsize=(8,5))
plt.title("Performance Comparison: VGG16 vs VGG19")
plt.ylabel("Score")
plt.xticks(rotation=0)
plt.show()
```



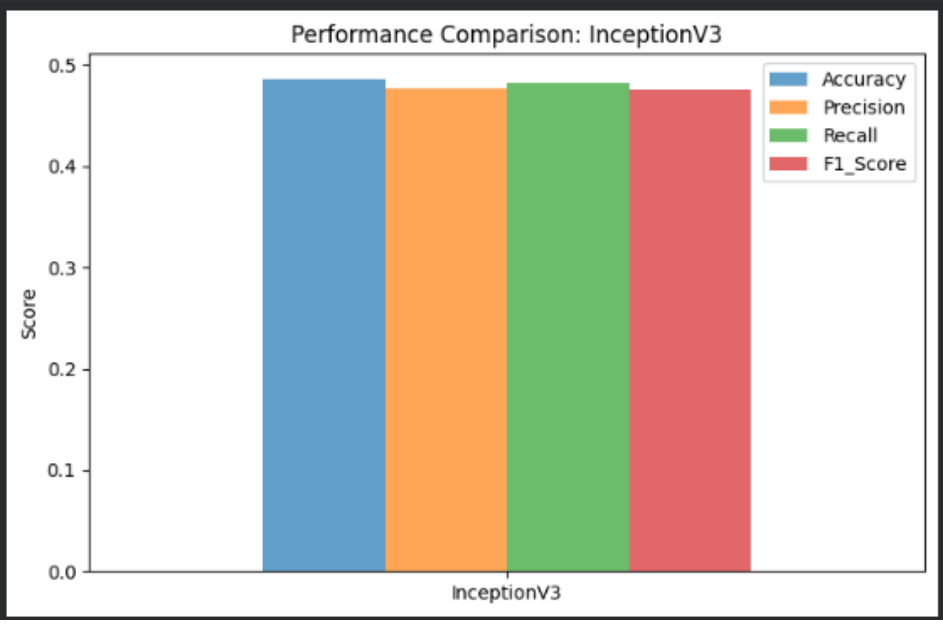
Optional




```
# Final performance table with InceptionV3

PretrainedModels = ['InceptionV3']

ErrorMetrics = [Accuracy, Precision, Recall, F1_Score]

ErrorMetrics_pd = pd.DataFrame(ErrorMetrics)
ErrorMetrics_pd.columns = PretrainedModels
ErrorMetrics_pd = ErrorMetrics_pd.T
ErrorMetrics_pd.columns = ['Accuracy','Precision','Recall','F1_Score']

ErrorMetrics_pd
```

	Accuracy	Precision	Recall	F1_Score
InceptionV3	0.4865	0.476522	0.481677	0.475654

```
ErrorMetrics_pd.plot.bar(width=0.7, alpha=0.7, figsize=(8,5))
plt.title('Performance Comparison: InceptionV3')
plt.ylabel('Score')
plt.xticks(rotation=0)
plt.show()
```



Lab 8

```python
# Create the modified LSTM model using SID 2128741, so parameter = 41 + 10 = 51

from tensorflow import keras
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
import matplotlib.pyplot as plt

lstm_parameter = 51

model_logbook = keras.Sequential([
    keras.layers.Input(shape=(50, 18)),
    keras.layers.LSTM(lstm_parameter, activation='relu'),
    keras.layers.Dense(2)
])

model_logbook.summary()
```

```
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
lstm_1 (LSTM)	(None, 51)	14,280
dense_1 (Dense)	(None, 2)	104

```
Total params: 14,384 (56.19 KB)
Trainable params: 14,384 (56.19 KB)
Non-trainable params: 0 (0.00 B)
```

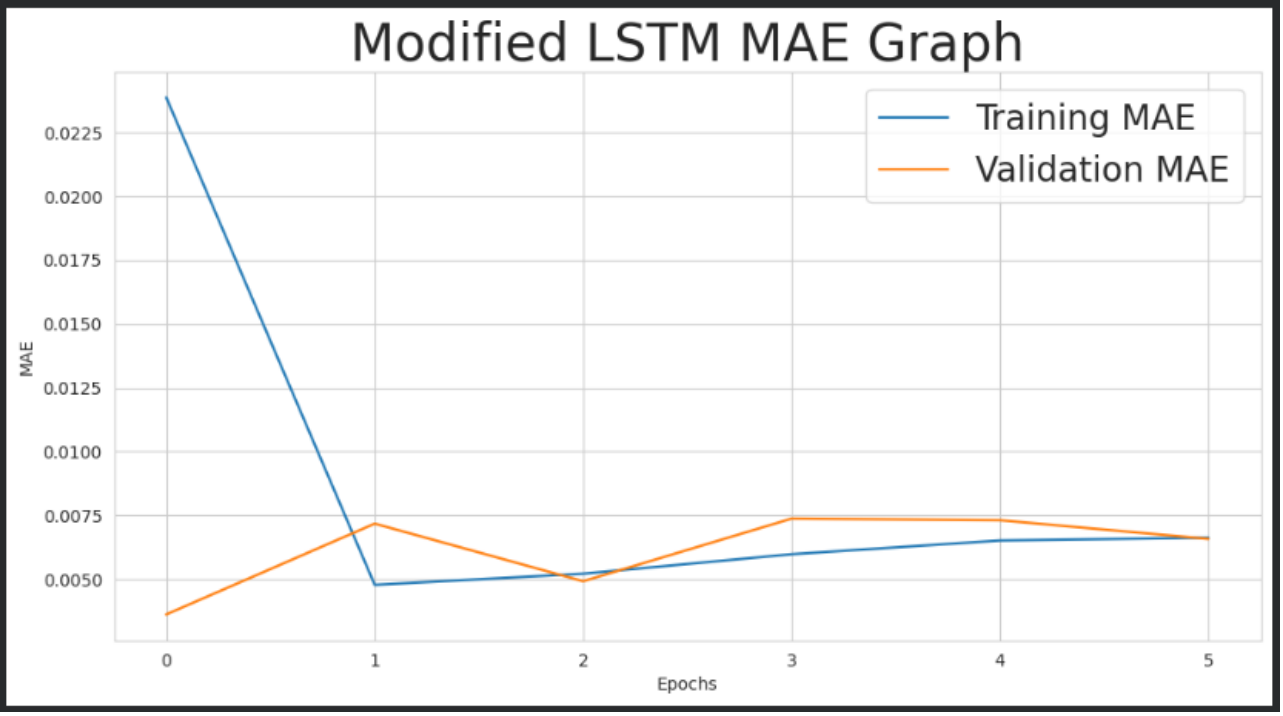
```python
# Compile and train the modified LSTM model with epochs = 10 and patience = 5

model_logbook.compile(optimizer='adam', loss='mse', metrics=['mae'])

es_logbook = EarlyStopping(monitor='val_loss', mode='min', patience=5, verbose=1)

mc_logbook = ModelCheckpoint(
    'best_model_LSTM_logbook.keras',
    monitor='val_loss',
    mode='min',
    verbose=1,
    save_best_only=True
)

history_logbook = model_logbook.fit(
    X_train,
    y_train,
    batch_size=20,
    epochs=10,
    validation_split=0.1,
    verbose=1,
    callbacks=[es_logbook, mc_logbook]
)
```

```
Epoch 1/10
1424/1425 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - loss: 0.1092 - mae: 0.0857
Epoch 1: val_loss improved from None to 0.00003, saving model to best_model_LSTM_logbook.keras

Epoch 1: finished saving model to best_model_LSTM_logbook.keras
1425/1425 ━━━━━━━━━━━━━━━━━━━━ 33s 22ms/step - loss: 0.0167 - mae: 0.0239 - val_loss: 2.7753e-05 - val_mae: 0.0036
Epoch 2/10
1425/1425 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 4.2616e-05 - mae: 0.0046
Epoch 2: val_loss did not improve from 0.00003
1425/1425 ━━━━━━━━━━━━━━━━━━━━ 27s 19ms/step - loss: 4.4804e-05 - mae: 0.0048 - val_loss: 7.1071e-05 - val_mae: 0.0072
Epoch 3/10
1424/1425 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 4.5624e-05 - mae: 0.0049
Epoch 3: val_loss did not improve from 0.00003
1425/1425 ━━━━━━━━━━━━━━━━━━━━ 41s 19ms/step - loss: 4.9966e-05 - mae: 0.0052 - val_loss: 3.8620e-05 - val_mae: 0.0049
Epoch 4/10
1425/1425 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 6.0936e-05 - mae: 0.0059
Epoch 4: val_loss did not improve from 0.00003
1425/1425 ━━━━━━━━━━━━━━━━━━━━ 28s 19ms/step - loss: 6.2912e-05 - mae: 0.0060 - val_loss: 6.8414e-05 - val_mae: 0.0074
Epoch 5/10
1424/1425 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - loss: 7.4235e-05 - mae: 0.0067
Epoch 5: val_loss did not improve from 0.00003
1425/1425 ━━━━━━━━━━━━━━━━━━━━ 43s 21ms/step - loss: 7.1194e-05 - mae: 0.0065 - val_loss: 7.8956e-05 - val_mae: 0.0073
Epoch 6/10
1424/1425 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 7.9367e-05 - mae: 0.0069
Epoch 6: val_loss did not improve from 0.00003
1425/1425 ━━━━━━━━━━━━━━━━━━━━ 28s 20ms/step - loss: 7.4034e-05 - mae: 0.0066 - val_loss: 6.5949e-05 - val_mae: 0.0066
Epoch 6: early stopping
```

```python
# Load the best saved model and evaluate test MSE and MAE

best_model_logbook = keras.models.load_model('best_model_LSTM_logbook.keras')

logbook_scores = best_model_logbook.evaluate(X_test, y_test, verbose=1)

print("Modified LSTM Test MSE:", logbook_scores[0])
print("Modified LSTM Test MAE:", logbook_scores[1])

try:
    print("Practical LSTM Test MSE:", scores[0])
    print("Practical LSTM Test MAE:", scores[1])
except:
    print("Practical LSTM scores variable is not available in this session.")
```

```
110/110 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 3.0865e-05 - mae: 0.0037
Modified LSTM Test MSE: 3.086531432927586e-05
Modified LSTM Test MAE: 0.0037354757077991962
Practical LSTM Test MSE: 1.79658891283907e-05
Practical LSTM Test MAE: 0.002956385025754571
```

```python
# Plot detailed MAE graph for training and validation

plt.figure(figsize=(12,6))
plt.plot(history_logbook.history['mae'], label='Training MAE')
plt.plot(history_logbook.history['val_mae'], label='Validation MAE')
plt.title('Modified LSTM MAE Graph')
plt.xlabel('Epochs')
plt.ylabel('MAE')
plt.legend()
plt.show()
```



Lab 9

```
# Set my SID-based training parameters for the optimised best LSTM model
Z = 1
my_epochs = Z + 30
my_batch_size = Z + 10

print("My epochs:", my_epochs)
print("My batch size:", my_batch_size)

# Stop training early if validation loss does not improve
es = EarlyStopping(monitor='val_loss', mode='min', patience=10, verbose=1, restore_best_weights=True)

# Save only the best optimised LSTM model during training
mc = ModelCheckpoint('my_optimised_best_LSTM_model.keras', monitor='val_loss', mode='min', verbose=1, save_best_only=True)
```

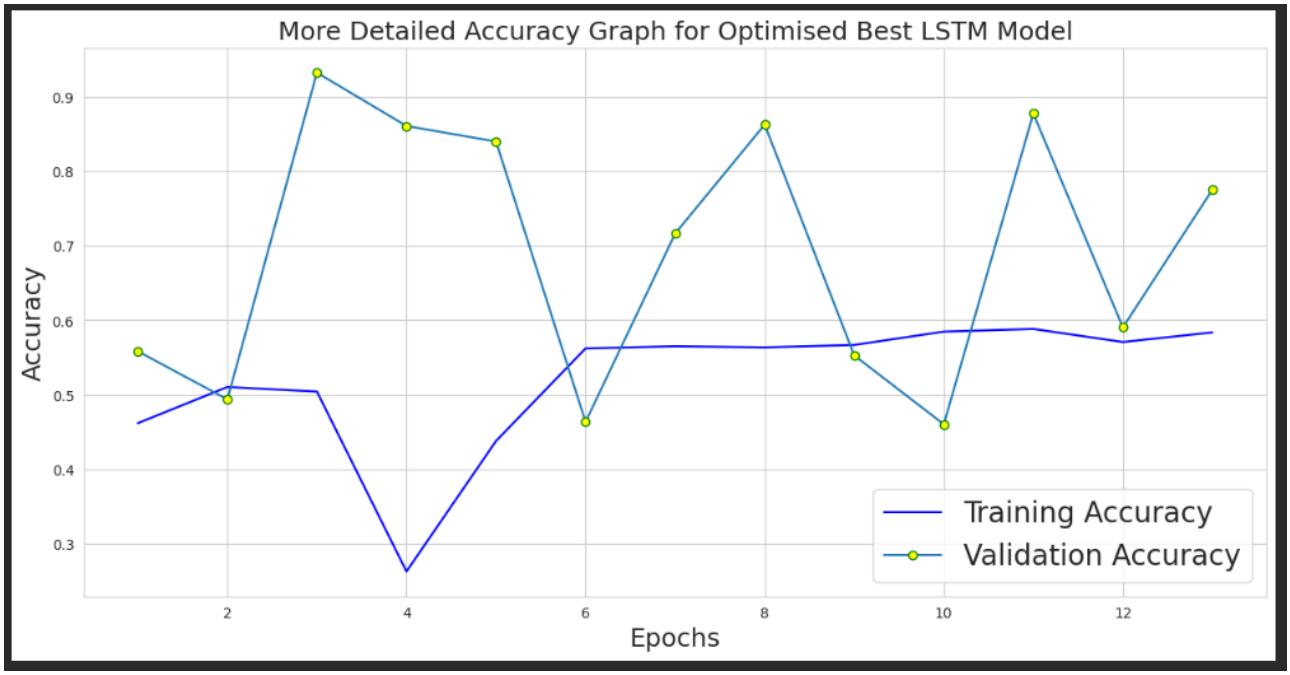
```
My epochs: 31
My batch size: 11
```

```
# Train the optimised best LSTM model using my SID-based epochs and batch size
history_optimised = best_model.fit(
    X_train,
    y_train,
    batch_size=my_batch_size,
    epochs=my_epochs,
    validation_split=0.2,
    shuffle=True,
    verbose=1,
    callbacks=[es, mc],
    class_weight=class_weight_dict
)
```

```
Epoch 1/31
783/783 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step - accuracy: 0.3961 - loss: 0.7130
Epoch 1: val_loss improved from None to 0.70506, saving model to my_optimised_best_LSTM_model.keras

Epoch 1: finished saving model to my_optimised_best_LSTM_model.keras
783/783 ━━━━━━━━━━━━━━━━━━━━ 36s 41ms/step - accuracy: 0.4618 - loss: 0.7058 - val_accuracy: 0.5583 - val_loss: 0.7051
Epoch 2/31
782/783 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step - accuracy: 0.4924 - loss: 0.6959
Epoch 2: val_loss did not improve from 0.70506
783/783 ━━━━━━━━━━━━━━━━━━━━ 33s 42ms/step - accuracy: 0.5103 - loss: 0.6912 - val_accuracy: 0.4937 - val_loss: 0.7214
Epoch 3/31
782/783 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step - accuracy: 0.5054 - loss: 0.6860
Epoch 3: val_loss improved from 0.70506 to 0.27051, saving model to my_optimised_best_LSTM_model.keras

Epoch 3: finished saving model to my_optimised_best_LSTM_model.keras
783/783 ━━━━━━━━━━━━━━━━━━━━ 31s 40ms/step - accuracy: 0.5042 - loss: 0.6949 - val_accuracy: 0.9326 - val_loss: 0.2705
Epoch 4/31
783/783 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step - accuracy: 0.2961 - loss: 0.7581
Epoch 4: val_loss did not improve from 0.27051
783/783 ━━━━━━━━━━━━━━━━━━━━ 34s 43ms/step - accuracy: 0.2626 - loss: 0.7335 - val_accuracy: 0.8610 - val_loss: 0.6585
Epoch 5/31
783/783 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step - accuracy: 0.4109 - loss: 0.7172
Epoch 5: val_loss did not improve from 0.27051
783/783 ━━━━━━━━━━━━━━━━━━━━ 32s 40ms/step - accuracy: 0.4378 - loss: 0.7134 - val_accuracy: 0.8401 - val_loss: 0.5619
Epoch 6/31
782/783 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step - accuracy: 0.5838 - loss: 0.6805
Epoch 6: val_loss did not improve from 0.27051
783/783 ━━━━━━━━━━━━━━━━━━━━ 42s 42ms/step - accuracy: 0.5622 - loss: 0.6922 - val_accuracy: 0.4635 - val_loss: 0.7538
Epoch 7/31
783/783 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step - accuracy: 0.6169 - loss: 0.6730
Epoch 7: val_loss did not improve from 0.27051
783/783 ━━━━━━━━━━━━━━━━━━━━ 42s 43ms/step - accuracy: 0.5651 - loss: 0.6839 - val_accuracy: 0.7164 - val_loss: 0.6523
Epoch 8/31
782/783 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step - accuracy: 0.5762 - loss: 0.6813
Epoch 8: val_loss did not improve from 0.27051
783/783 ━━━━━━━━━━━━━━━━━━━━ 33s 42ms/step - accuracy: 0.5636 - loss: 0.6872 - val_accuracy: 0.8629 - val_loss: 0.5404
Epoch 9/31
783/783 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step - accuracy: 0.6048 - loss: 0.6742
Epoch 9: val_loss did not improve from 0.27051
783/783 ━━━━━━━━━━━━━━━━━━━━ 43s 45ms/step - accuracy: 0.5668 - loss: 0.6742 - val_accuracy: 0.5528 - val_loss: 0.6916
Epoch 10/31
782/783 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step - accuracy: 0.5750 - loss: 0.6630
Epoch 10: val_loss did not improve from 0.27051
783/783 ━━━━━━━━━━━━━━━━━━━━ 32s 40ms/step - accuracy: 0.5847 - loss: 0.6624 - val_accuracy: 0.4603 - val_loss: 0.7330
Epoch 11/31
782/783 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step - accuracy: 0.5543 - loss: 0.6747
Epoch 11: val_loss did not improve from 0.27051
783/783 ━━━━━━━━━━━━━━━━━━━━ 32s 41ms/step - accuracy: 0.5883 - loss: 0.6559 - val_accuracy: 0.8777 - val_loss: 0.4595
Epoch 12/31
783/783 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step - accuracy: 0.5867 - loss: 0.6475
Epoch 12: val_loss did not improve from 0.27051
783/783 ━━━━━━━━━━━━━━━━━━━━ 32s 41ms/step - accuracy: 0.5708 - loss: 0.6501 - val_accuracy: 0.5904 - val_loss: 0.6487
Epoch 13/31
783/783 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step - accuracy: 0.6042 - loss: 0.6462
Epoch 13: val_loss did not improve from 0.27051
783/783 ━━━━━━━━━━━━━━━━━━━━ 32s 41ms/step - accuracy: 0.5837 - loss: 0.6508 - val_accuracy: 0.7755 - val_loss: 0.5433
Epoch 13: early stopping
Restoring model weights from the end of the best epoch: 3.
```

```
# Load the best saved optimised LSTM model
optimised_LSTM_saved_best_model = keras.models.load_model('my_optimised_best_LSTM_model.keras')

# Evaluate the optimised best LSTM model on the test dataset
test_loss, test_accuracy = optimised_LSTM_saved_best_model.evaluate(X_test, y_test, verbose=1)

print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)

38/38 ━━━━━━━━━━━━━━━━━━━━ 1s 13ms/step - accuracy: 0.8854 - loss: 0.3579
Test Loss: 0.3578535318374634
Test Accuracy: 0.8853556513786316
```

```
38/38 ━━━━━━━━━━━━━━━━━━━━ 2s 31ms/step
              precision    recall  f1-score   support

         0.0       0.89      1.00      0.94      1058
         1.0       0.00      0.00      0.00       137

    accuracy                           0.89      1195
   macro avg       0.44      0.50      0.47      1195
weighted avg       0.78      0.89      0.83      1195

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```


More Detailed Accuracy Graph for Optimised Best LSTM Model
Accuracy
Epochs
0.9
0.8
0.7
0.6
0.5
0.4
0.3
2
4
6
8
10
12
Training Accuracy
Validation Accuracy

Lab 10

Simple Encoder:

```
# Print the Encoder model summary for the lab logbook
encoder.summary()
```

Model: "sequential_3"

Layer (type)	Output Shape	Param #
flatten_1 (Flatten)	(None, 784)	0
gaussian_noise_1 (GaussianNoise)	(None, 784)	0
dense_10 (Dense)	(None, 400)	314,000
dense_11 (Dense)	(None, 200)	80,200
dense_12 (Dense)	(None, 100)	20,100
dense_13 (Dense)	(None, 50)	5,050
dense_14 (Dense)	(None, 25)	1,275

Total params: 420,625 (1.60 MB)
Trainable params: 420,625 (1.60 MB)
Non-trainable params: 0 (0.00 B)

```
# Print the Decoder model summary for the lab logbook
decoder.summary()
```

Model: "sequential_4"

Layer (type)	Output Shape	Param #
dense_15 (Dense)	(None, 50)	1,300
dense_16 (Dense)	(None, 100)	5,100
dense_17 (Dense)	(None, 200)	20,200
dense_18 (Dense)	(None, 400)	80,400
dense_19 (Dense)	(None, 784)	314,384
reshape_1 (Reshape)	(None, 28, 28)	0

Total params: 421,384 (1.61 MB)
Trainable params: 421,384 (1.61 MB)
Non-trainable params: 0 (0.00 B)

```
# Print the Basic Autoencoder model summary for the lab logbook
autoencoder.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
sequential (Sequential)	(None, 25)	420,625
sequential_1 (Sequential)	(None, 28, 28)	421,384

Total params: 2,526,029 (9.64 MB)
Trainable params: 842,009 (3.21 MB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 1,684,020 (6.42 MB)

CNN Encoder:

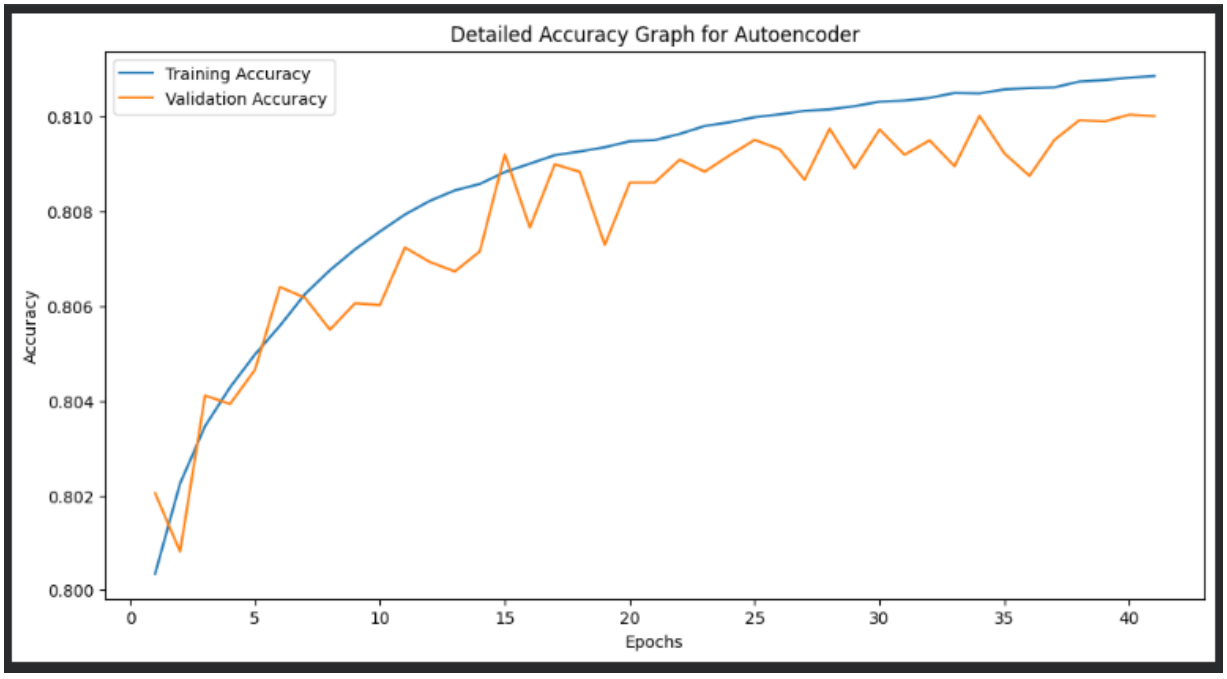
```
# Train the autoencoder with SID-based epochs (Z+40)
Z = 1
my_epochs = Z + 40

x_train_fast = x_train[:10000]

history_2 = autoencoder_2.fit(
    x_train_fast,
    x_train_fast,
    epochs=my_epochs,
    batch_size=256,
    validation_split=0.2,
    shuffle=True,
    verbose=1
)
```

```
Epoch 13/41
32/32 ━━━━━━━━━━━━━━━━━━━━ 9s 261ms/step - accuracy: 0.8084 - loss: 0.1205 - val_accuracy: 0.8067 - val_loss: 0.1220
Epoch 14/41
32/32 ━━━━━━━━━━━━━━━━━━━━ 10s 258ms/step - accuracy: 0.8086 - loss: 0.1195 - val_accuracy: 0.8072 - val_loss: 0.1208
Epoch 15/41
32/32 ━━━━━━━━━━━━━━━━━━━━ 10s 305ms/step - accuracy: 0.8088 - loss: 0.1182 - val_accuracy: 0.8092 - val_loss: 0.1212
Epoch 16/41
32/32 ━━━━━━━━━━━━━━━━━━━━ 10s 307ms/step - accuracy: 0.8090 - loss: 0.1174 - val_accuracy: 0.8077 - val_loss: 0.1189
Epoch 17/41
32/32 ━━━━━━━━━━━━━━━━━━━━ 9s 272ms/step - accuracy: 0.8092 - loss: 0.1163 - val_accuracy: 0.8090 - val_loss: 0.1186
Epoch 18/41
32/32 ━━━━━━━━━━━━━━━━━━━━ 10s 262ms/step - accuracy: 0.8093 - loss: 0.1157 - val_accuracy: 0.8088 - val_loss: 0.1178
Epoch 19/41
32/32 ━━━━━━━━━━━━━━━━━━━━ 11s 287ms/step - accuracy: 0.8094 - loss: 0.1153 - val_accuracy: 0.8073 - val_loss: 0.1174
Epoch 20/41
32/32 ━━━━━━━━━━━━━━━━━━━━ 11s 309ms/step - accuracy: 0.8095 - loss: 0.1144 - val_accuracy: 0.8086 - val_loss: 0.1163
Epoch 21/41
32/32 ━━━━━━━━━━━━━━━━━━━━ 10s 309ms/step - accuracy: 0.8095 - loss: 0.1145 - val_accuracy: 0.8086 - val_loss: 0.1158
Epoch 22/41
32/32 ━━━━━━━━━━━━━━━━━━━━ 10s 289ms/step - accuracy: 0.8096 - loss: 0.1135 - val_accuracy: 0.8091 - val_loss: 0.1152
Epoch 23/41
32/32 ━━━━━━━━━━━━━━━━━━━━ 9s 266ms/step - accuracy: 0.8098 - loss: 0.1127 - val_accuracy: 0.8088 - val_loss: 0.1152
Epoch 24/41
32/32 ━━━━━━━━━━━━━━━━━━━━ 10s 308ms/step - accuracy: 0.8099 - loss: 0.1122 - val_accuracy: 0.8092 - val_loss: 0.1142
Epoch 25/41
32/32 ━━━━━━━━━━━━━━━━━━━━ 10s 305ms/step - accuracy: 0.8100 - loss: 0.1116 - val_accuracy: 0.8095 - val_loss: 0.1138
Epoch 26/41
32/32 ━━━━━━━━━━━━━━━━━━━━ 9s 280ms/step - accuracy: 0.8101 - loss: 0.1113 - val_accuracy: 0.8093 - val_loss: 0.1134
Epoch 27/41
32/32 ━━━━━━━━━━━━━━━━━━━━ 9s 290ms/step - accuracy: 0.8101 - loss: 0.1109 - val_accuracy: 0.8087 - val_loss: 0.1131
Epoch 28/41
32/32 ━━━━━━━━━━━━━━━━━━━━ 10s 306ms/step - accuracy: 0.8102 - loss: 0.1104 - val_accuracy: 0.8097 - val_loss: 0.1125
Epoch 29/41
32/32 ━━━━━━━━━━━━━━━━━━━━ 10s 329ms/step - accuracy: 0.8102 - loss: 0.1102 - val_accuracy: 0.8089 - val_loss: 0.1124
Epoch 30/41
32/32 ━━━━━━━━━━━━━━━━━━━━ 9s 271ms/step - accuracy: 0.8103 - loss: 0.1096 - val_accuracy: 0.8097 - val_loss: 0.1121
Epoch 31/41
32/32 ━━━━━━━━━━━━━━━━━━━━ 11s 274ms/step - accuracy: 0.8103 - loss: 0.1094 - val_accuracy: 0.8092 - val_loss: 0.1122
Epoch 32/41
32/32 ━━━━━━━━━━━━━━━━━━━━ 10s 320ms/step - accuracy: 0.8104 - loss: 0.1091 - val_accuracy: 0.8095 - val_loss: 0.1118
Epoch 33/41
32/32 ━━━━━━━━━━━━━━━━━━━━ 10s 309ms/step - accuracy: 0.8105 - loss: 0.1086 - val_accuracy: 0.8090 - val_loss: 0.1108
Epoch 34/41
32/32 ━━━━━━━━━━━━━━━━━━━━ 9s 265ms/step - accuracy: 0.8105 - loss: 0.1085 - val_accuracy: 0.8100 - val_loss: 0.1109
Epoch 35/41
32/32 ━━━━━━━━━━━━━━━━━━━━ 10s 268ms/step - accuracy: 0.8106 - loss: 0.1081 - val_accuracy: 0.8092 - val_loss: 0.1102
Epoch 36/41
32/32 ━━━━━━━━━━━━━━━━━━━━ 10s 307ms/step - accuracy: 0.8106 - loss: 0.1078 - val_accuracy: 0.8088 - val_loss: 0.1103
Epoch 37/41
32/32 ━━━━━━━━━━━━━━━━━━━━ 10s 311ms/step - accuracy: 0.8106 - loss: 0.1077 - val_accuracy: 0.8095 - val_loss: 0.1092
Epoch 38/41
32/32 ━━━━━━━━━━━━━━━━━━━━ 9s 268ms/step - accuracy: 0.8107 - loss: 0.1070 - val_accuracy: 0.8099 - val_loss: 0.1093
Epoch 39/41
32/32 ━━━━━━━━━━━━━━━━━━━━ 10s 259ms/step - accuracy: 0.8108 - loss: 0.1069 - val_accuracy: 0.8099 - val_loss: 0.1090
Epoch 40/41
32/32 ━━━━━━━━━━━━━━━━━━━━ 11s 287ms/step - accuracy: 0.8108 - loss: 0.1064 - val_accuracy: 0.8100 - val_loss: 0.1086
Epoch 41/41
32/32 ━━━━━━━━━━━━━━━━━━━━ 10s 300ms/step - accuracy: 0.8109 - loss: 0.1064 - val_accuracy: 0.8100 - val_loss: 0.1088
```

```
Model: "sequential_3"
```

Layer (type)	Output Shape	Param #
sequential_1 (Sequential)	(None, 30)	167,166
sequential_2 (Sequential)	(None, 28, 28, 1)	14,049

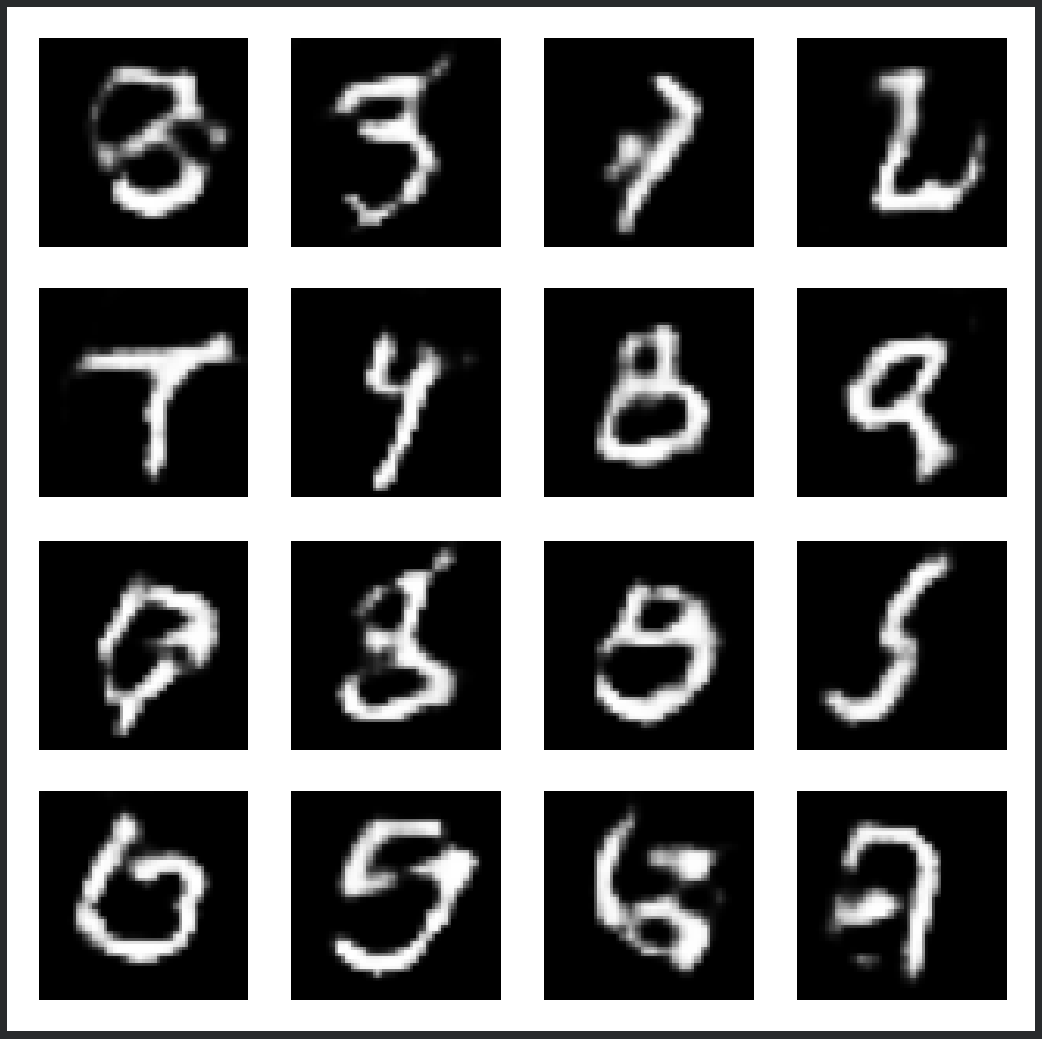
```
Total params: 543,647 (2.07 MB)
Trainable params: 181,215 (707.87 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 362,432 (1.38 MB)
```


Detailed Accuracy Graph for Autoencoder
Training Accuracy
Validation Accuracy
Accuracy
0.810
0.808
0.806
0.804
0.802
0.800
0
5
10
15
20
25
30
35
40
Epochs

313/313
6s 19ms/step
Original
Decoded

Lab 11

```
train(train_dataset, EPOCHS)
```



```
Time for epoch 41 is 13.349193334579468 sec
```

Lab 12

```python
# Import EarlyStopping and ModelCheckpoint
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.callbacks import ModelCheckpoint
```

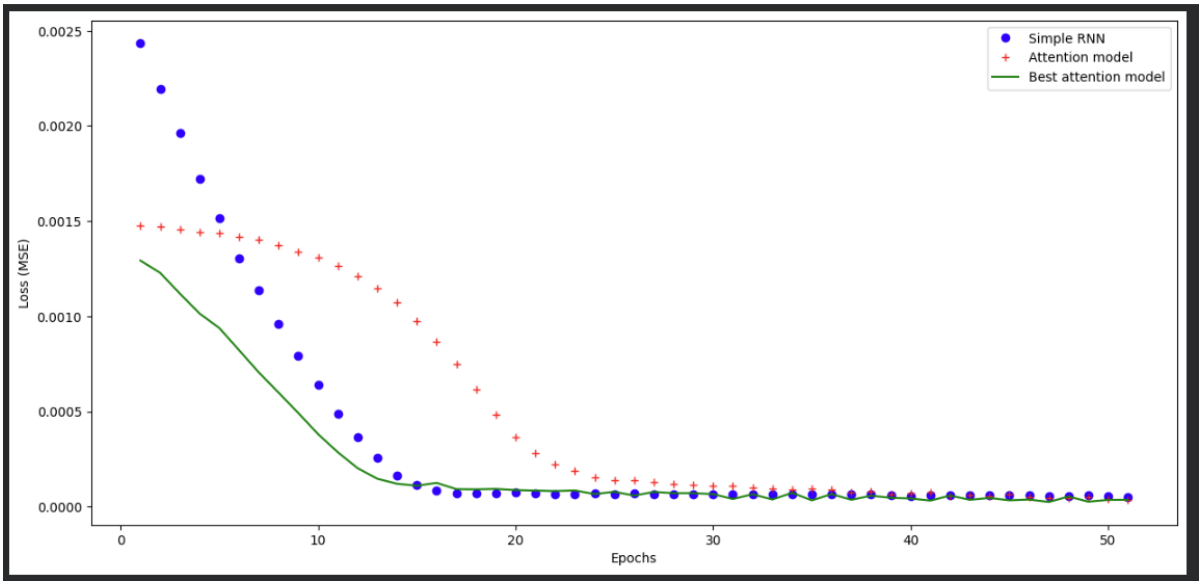
```python
# Modify parameters using SID (Z = 1)
Z = 1

time_steps = 20  # fixed as per requirement
hidden_units = 4  # fixed as per requirement

epochs = Z + 50  # 51
patience = Z + 10  # 11

print("epochs =", epochs)
print("patience =", patience)
print("time_steps =", time_steps)
print("hidden_units =", hidden_units)
```

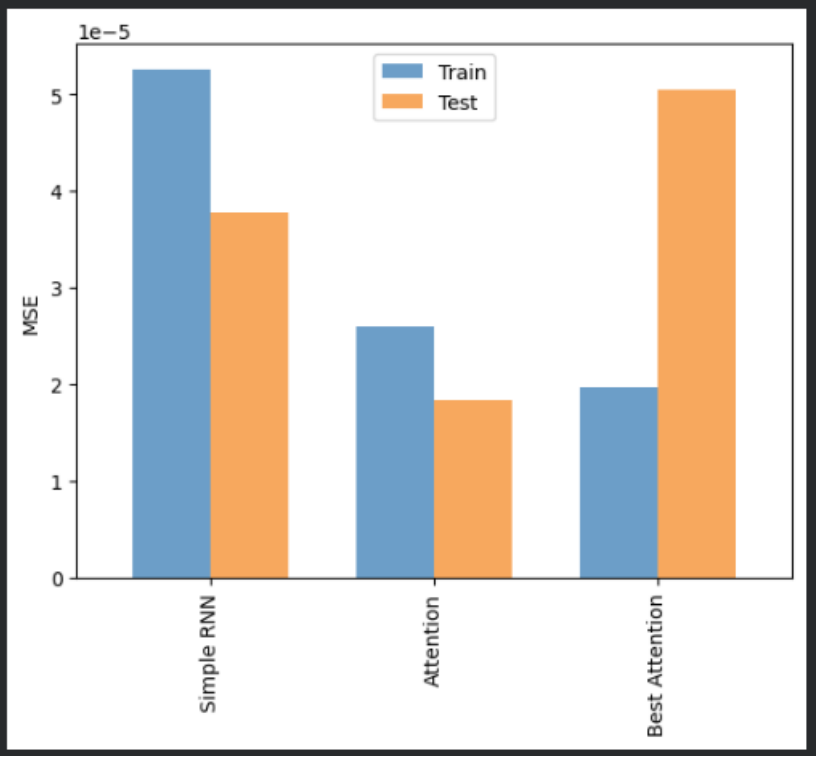
```
epochs = 51
patience = 11
time_steps = 20
hidden_units = 4
```

```python
# Define EarlyStopping and ModelCheckpoint
early_stop = EarlyStopping(
    monitor='loss',
    mode='min',
    patience=patience,
    verbose=1,
    restore_best_weights=True
)

model_ckpt = ModelCheckpoint(
    'best_attention_model.keras',
    monitor='loss',
    mode='min',
    verbose=1,
    save_best_only=True
)
```

```
826/826 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 5.7540e-05
Epoch 39/51
824/826 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 1.3464e-05
Epoch 39: loss did not improve from 0.00003
826/826 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - loss: 4.6665e-05
Epoch 40/51
804/826 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 2.3381e-05
Epoch 40: loss did not improve from 0.00003
826/826 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - loss: 4.3187e-05
Epoch 41/51
821/826 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 1.1520e-05
Epoch 41: loss improved from 0.00003 to 0.00003, saving model to best_attention_model.keras

Epoch 41: finished saving model to best_attention_model.keras
826/826 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - loss: 3.1982e-05
Epoch 42/51
805/826 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 1.5822e-05
Epoch 42: loss did not improve from 0.00003
826/826 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - loss: 5.8461e-05
Epoch 43/51
821/826 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 3.2324e-05
Epoch 43: loss did not improve from 0.00003
826/826 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - loss: 3.6199e-05
Epoch 44/51
820/826 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 3.6838e-05
Epoch 44: loss did not improve from 0.00003
826/826 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 4.5900e-05
Epoch 45/51
826/826 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 4.2145e-05
Epoch 45: loss did not improve from 0.00003
826/826 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - loss: 3.3455e-05
Epoch 46/51
826/826 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 4.7212e-05
Epoch 46: loss did not improve from 0.00003
826/826 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - loss: 3.7475e-05
Epoch 47/51
822/826 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 1.6017e-05
Epoch 47: loss improved from 0.00003 to 0.00002, saving model to best_attention_model.keras

Epoch 47: finished saving model to best_attention_model.keras
826/826 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - loss: 2.4844e-05
Epoch 48/51
826/826 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 4.9547e-05
Epoch 48: loss did not improve from 0.00002
826/826 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - loss: 5.3199e-05
Epoch 49/51
825/826 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 1.7004e-05
Epoch 49: loss did not improve from 0.00002
826/826 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 2.6346e-05
Epoch 50/51
823/826 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 1.5105e-05
Epoch 50: loss did not improve from 0.00002
826/826 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - loss: 3.5717e-05
Epoch 51/51
819/826 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 7.5942e-05
Epoch 51: loss did not improve from 0.00002
826/826 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - loss: 3.6026e-05
Restoring model weights from the end of the best epoch: 47.
```

```
26/26 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 1.9719e-05
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 5.0560e-05
Train set MSE = 1.971872006834019e-05
Test set MSE = 5.0559581723064184e-05
```




```python
# Create histogram for performance comparison
import pandas as pd

ErrorMetrics = [
    [train_mse, test_mse],                  # Simple RNN
    [train_mse_attn, test_mse_attn],        # Attention model
    [train_mse_new, test_mse_new]           # Best attention model
]

ErrorMetrics_pd = pd.DataFrame(ErrorMetrics)
ErrorMetrics_pd.columns = ['Train', 'Test']
ErrorMetrics_pd.index = ['Simple RNN', 'Attention', 'Best Attention']

ErrorMetrics_pd.plot.bar(width=0.7, alpha=0.7)
plt.ylabel('MSE')
plt.show()
```